```
// One merged endpoint for all admin
actions, to stay under Vercel's function
limit.
// GET/POST/PUT/DELETE /api/admin?
resource=tasks|users|withdrawals|moderation
|broadcast|referral-audit|ledger
const { supabase } =
require('../lib/supabase');
const { requireAdmin } =
require('../lib/adminAuth');

module.exports = async (req, res) => {
  try {
    if (!requireAdmin(req, res)) return;

    const resource =
String(req.query.resource || '').trim();

    // ===== TASKS =====
    if (resource === 'tasks') {
      if (req.method === 'GET') {
        const { data, error } = await
supabase.from('tasks').select('*').order('p
latform');
        if (error) throw error;
        return res.status(200).json({
tasks: data });
      }
      if (req.method === 'POST') {
        const { title, points, platform } =
```

```
req.body || {};
        if (!title || points == null)
return res.status(400).json({ error: 'title
and points are required' });
        const { data, error } = await
supabase
          .from('tasks')
          .insert({ title, points,
platform: platform || 'general', active:
true })
          .select()
          .single();
        if (error) throw error;
        return res.status(200).json({ ok:
true, id: data.id });
      }
      if (req.method === 'PUT') {
        const { id, ...updates } = req.body
|| {};
        if (!id) return
res.status(400).json({ error: 'id is
required' });
        const { error } = await
supabase.from('tasks').update(updates).eq('
id', id);
        if (error) throw error;
        return res.status(200).json({ ok:
true });
      }
      if (req.method === 'DELETE') {
        const { id } = req.body || {};
        if (!id) return
res.status(400).json({ error: 'id is
required' });
```

```
        const { error } = await
supabase.from('tasks').delete().eq('id',
id);
        if (error) throw error;
        return res.status(200).json({ ok:
true });
      }
    }

    // ===== USERS (lookup) =====
    if (resource === 'users') {
      const userId =
String(req.query.userId || '').trim();
      if (!userId) return
res.status(400).json({ error: 'userId is
required' });
      const { data: user, error } = await
supabase.from('users').select('*').eq('id',
userId).single();
      if (error && error.code ===
'PGRST116') return res.status(404).json({
error: 'User not found' });
      if (error) throw error;
      return res.status(200).json({ user
});
    }

    // ===== WITHDRAWALS =====
    if (resource === 'withdrawals') {
      if (req.method === 'GET') {
        const { data, error } = await
supabase
          .from('withdrawals')
          .select('*')
```

```
          .eq('status', 'pending')
          .order('created_at', { ascending:
false });
        if (error) throw error;
        return res.status(200).json({
withdrawals: data });
      }
      if (req.method === 'POST') {
        const { id, action } = req.body ||
{};
        if (!id || !['approve',
'reject'].includes(action)) {
          return res.status(400).json({
error: 'id and action (approve/reject) are
required' });
        }
        const { data, error } = await
supabase.rpc('resolve_withdrawal', {
p_withdrawal_id: id, p_action: action });
        if (error) throw error;
        if (data && data.error) return
res.status(400).json({ error: data.error
});
        return res.status(200).json({ ok:
true });
      }
    }

    // ===== MODERATION (ban/unban) =====
    if (resource === 'moderation') {
      if (req.method === 'GET') {
        const userId =
String(req.query.userId || '').trim();
        if (!userId) return
```

```
res.status(400).json({ error: 'userId is
required' });
        const { data, error } = await
supabase.from('users').select('id,banned').
eq('id', userId).single();
        if (error && error.code ===
'PGRST116') return res.status(404).json({
error: 'User not found' });
        if (error) throw error;
        return res.status(200).json(data);
      }
      if (req.method === 'POST') {
        const { userId, banned } = req.body
|| {};
        if (!userId || typeof banned !==
'boolean') {
          return res.status(400).json({
error: 'userId and banned (true/false) are
required' });
        }
        const { error } = await
supabase.from('users').update({ banned
}).eq('id', userId);
        if (error) throw error;
        return res.status(200).json({ ok:
true });
      }
    }

    // ===== BROADCAST =====
    if (resource === 'broadcast') {
      if (req.method !== 'POST') return
res.status(405).json({ error: 'POST only'
});
```

```
      const { message } = req.body || {};
      if (!message) return
res.status(400).json({ error: 'message is
required' });

      const botToken =
process.env.TELEGRAM_BOT_TOKEN;
      if (!botToken) return
res.status(500).json({ error: 'Bot token
not configured' });

      const { data: users, error } = await
supabase.from('users').select('id').eq('ban
ned', false);
      if (error) throw error;

      let sent = 0;
      let failed = 0;
      for (const u of users) {
        try {
          const url =
`https://api.telegram.org/bot${botToken}/se
ndMessage`;
          const r = await fetch(url, {
            method: 'POST',
            headers: { 'Content-Type':
'application/json' },
            body: JSON.stringify({ chat_id:
u.id, text: message }),
          });
          if (r.ok) sent += 1; else failed
+= 1;
        } catch (e) {
          failed += 1;
```

```
        }
        await new Promise((resolve) =>
setTimeout(resolve, 40));
      }
      return res.status(200).json({ ok:
true, sent, failed, total: users.length });
    }

    // ===== REFERRAL AUDIT =====
    if (resource === 'referral-audit') {
      const userId =
String(req.query.userId || '').trim();
      if (!userId) return
res.status(400).json({ error: 'userId is
required' });

      const { data: user, error } = await
supabase.from('users').select('*').eq('id',
userId).single();
      if (error && error.code ===
'PGRST116') return res.status(404).json({
error: 'User not found' });
      if (error) throw error;

      const { data: referredUsers, error:
refErr } = await supabase
        .from('users')

.select('id,referral_stage,balance,created_
at')
        .eq('referred_by', userId);
      if (refErr) throw refErr;

      return res.status(200).json({
```

```
        userId,
        referredBy: user.referred_by,
        referralCount: user.referral_count,
        referredUsers,
      });
    }

    // ===== LEDGER =====
    if (resource === 'ledger') {
      const userId =
String(req.query.userId || '').trim();
      if (!userId) return
res.status(400).json({ error: 'userId is
required' });

      const { data, error } = await
supabase
        .from('transactions')
        .select('*')
        .eq('user_id', userId)
        .order('created_at', { ascending:
false })
        .limit(100);
      if (error) throw error;

      return res.status(200).json({
transactions: data });
    }

    return res.status(400).json({ error:
'Unknown resource' });
  } catch (err) {
    console.error('admin.js error:', err);
    return res.status(500).json({ error:
```

```
'Internal error' });
  }
};
```